

PROYECTO ACADÉMICO · ARQUITECTURA SERVERLESS

Procesamiento automático de imágenes en AWS

Diseño orientado a eventos con Amazon S3, AWS Lambda, TypeScript, Node.js y Sharp

Universidad: Universidad Mariano Gálvez de Guatemala

Estudiante: Javier Guerra

Carné: [COMPLETAR]

Curso: [COMPLETAR]

Docente: [COMPLETAR]

Fecha: 4 de septiembre de 2026

INFORME CON EVIDENCIAS SIMULADAS

Nota obligatoria sobre las evidencias

Simulación académica — sin ejecución en AWS

Las capturas de Amazon S3, AWS Lambda, IAM y CloudWatch incluidas en este documento son recreaciones visuales. No fueron obtenidas desde una cuenta AWS activa y no deben presentarse como prueba de un despliegue real.

La cuenta AWS no pudo recuperarse. Para mantener la utilidad técnica del entregable se separaron cuidadosamente dos tipos de evidencia:

Validación real local

El código TypeScript compiló correctamente. Sharp procesó archivos JPEG y PNG reales en la computadora y se verificaron formato, dimensiones y tamaño.

Infraestructura simulada

Las pantallas de S3, Lambda, IAM y CloudWatch representan cómo se vería una configuración correcta usando nombres y mensajes coherentes con el proyecto.

Cómo sustituir esta simulación

Cuando exista acceso a AWS, se debe seguir el README, desplegar el ZIP, repetir la prueba y reemplazar las capturas 2 a 12 por imágenes reales de la consola. Las capturas 1, 13, 14, 15 y 16 ya reflejan archivos procesados realmente por Sharp en el entorno local.

Integridad académica: todas las recreaciones poseen una marca visible “SIMULACIÓN ACADÉMICA · SIN AWS REAL”.

Índice

1. Introducción	4	8. Permisos y seguridad	10
2. Objetivos	4	9. Logs y observabilidad	11
3. Arquitectura propuesta	5	10. Pruebas y resultados	12
4. Recursos utilizados	6	11. Ciclos e idempotencia	13
5. Implementación	7	12. Conclusiones	14
6. Configuración AWS propuesta	8	13. Repositorio y referencias	14
7. Dimensiones y Sharp	9	Anexo: 16 evidencias	15–30

Resumen ejecutivo

Se desarrolló una función en TypeScript que recibe eventos de Amazon S3, decodifica las keys, limita el procesamiento al prefijo `originals/`, descarga JPEG o PNG con AWS SDK v3, redimensiona con Sharp y guarda un objeto relacionado en `resized/`. El nombre determinista hace segura la repetición del evento.

La compilación local, la auditoría de dependencias y el procesamiento de imágenes fueron ejecutados realmente. La configuración cloud se documentó y se representó mediante capturas simuladas debido a la indisponibilidad de la cuenta AWS.

Introducción

El procesamiento manual de imágenes requiere intervención humana, consume tiempo y dificulta mantener dimensiones uniformes. Una arquitectura orientada a eventos permite reaccionar inmediatamente a una carga y ejecutar una transformación sin mantener servidores encendidos.

Amazon S3 actúa como almacenamiento y productor del evento; AWS Lambda aporta cómputo bajo demanda; Sharp realiza la transformación; IAM controla permisos; y CloudWatch centraliza logs. TypeScript facilita una implementación modular, legible y con validación estática.

Objetivo general

Diseñar e implementar una solución serverless que redimensione automáticamente imágenes JPEG y PNG al almacenarlas en el prefijo `originals/` de un bucket S3, preservando el original y guardando el resultado en `resized/`.

Objetivos específicos

- Procesar todos los registros contenidos en un evento S3.
- Obtener dimensiones desde `IMAGE_WIDTH` y `IMAGE_HEIGHT`.
- Conservar el tipo JPEG o PNG y un `Content-Type` correcto.
- Evitar ciclos mediante filtro de infraestructura y validación defensiva.
- Aplicar mínimo privilegio para lectura, escritura y logs.
- Producir salidas idempotentes y observables.

Alcance: el entregable incluye código, política IAM, empaquetado Linux x64, documentación y pruebas locales reales.

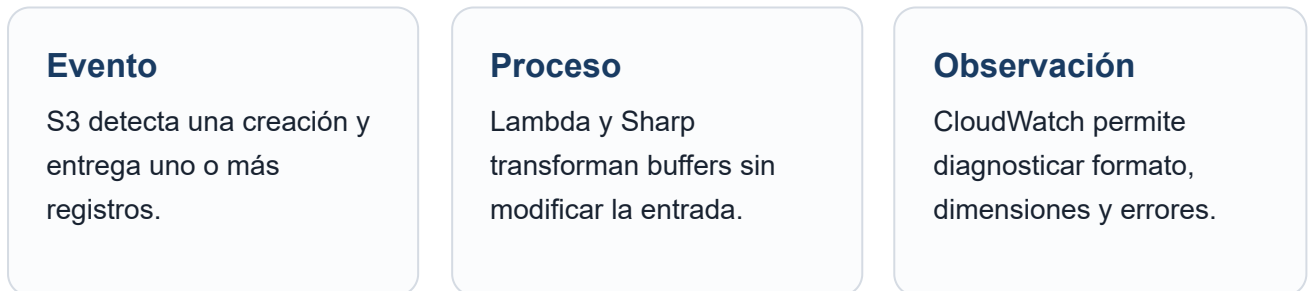
Arquitectura orientada a eventos



Secuencia

1. El usuario carga un objeto dentro de `originals/`.
2. S3 emite `s3:ObjectCreated:*` únicamente si la key coincide con ese prefijo.
3. Lambda identifica bucket y key, decodifica caracteres y valida extensión.
4. `GetObject` descarga el archivo a memoria.
5. Sharp comprueba el formato real, auto-orienta y redimensiona sin deformar.
6. `PutObject` guarda una key determinista en `resized/`.
7. Los mensajes de ejecución se escriben en CloudWatch Logs.

Separación de responsabilidades



Tecnologías y responsabilidades

Recurso	Uso dentro de la solución	Configuración principal
Amazon S3	Conserva entrada y salida como objetos privados.	<code>originals/</code> y <code>resized/</code>
AWS Lambda	Ejecuta el handler cuando S3 publica el evento.	Node.js 22.x, x86_64, 1024 MB, 30 s
Sharp 0.35.4	Lee metadata, auto-orienta y redimensiona.	Linux x64 + glibc
AWS SDK v3	Implementa <code>GetObject</code> y <code>PutObject</code> .	<code>@aws-sdk/client-s3</code>
IAM	Limita acceso por acción y prefijo.	Rol de ejecución de Lambda
CloudWatch Logs	Registra inicio, descarga, resize, guardado y errores.	<code>/aws/lambda/umg-image-resizer</code>
TypeScript 7	Tipado estricto y compilación a JavaScript.	Target ES2022, Node16 modules
GitHub	Control de versiones y entrega.	Rama <code>main</code>

Estructura modular

```
src/  
├─ index.ts      # Orquestación, lotes, logs y errores  
├─ config.ts     # Variables de entorno  
├─ imageProcessor.ts # Sharp y validación del formato real  
├─ s3Service.ts  # AWS SDK v3  
└─ utils.ts      # Prefijos, keys y Content-Type
```

Implementación de la función

El handler recorre secuencialmente `event.Records`. Si un registro falla, el error se conserva y se continúa con los demás. Al final se lanza un `AggregateError` para que una invocación parcialmente fallida no aparente éxito.

```
export async function handler(event: S3Event): Promise<void> {
  const results: RecordResult[] = [];
  const failures: Error[] = [];

  for (const [index, record] of event.Records.entries()) {
    try {
      results.push(await processRecord(record, index + 1));
    } catch (error: unknown) {
      console.error("Error al procesar registro S3.", ...);
      failures.push(error instanceof Error ? error : new Error(String(error)));
    }
  }

  if (failures.length > 0) throw new AggregateError(failures);
}
```

Construcción de la salida

`originals/viaje/Foto.JPEG` se transforma determinísticamente en `resized/viaje/Foto-resized.jpeg`. No se usan fechas ni identificadores aleatorios.

Validaciones

- Decodificación de `+` y caracteres URL-encoded.
- Prefijo obligatorio `originals/`.
- Extensiones `.jpg`, `.jpeg` y `.png`.
- Comparación entre extensión y formato binario detectado por Sharp.
- Dimensiones enteras, positivas y menores o iguales a 10 000.

Configuración de infraestructura

Estado: esta configuración se documenta y simula; no fue aplicada a una cuenta AWS.

Amazon S3

- Bucket de ejemplo: `umg-serverless-images-demo-jguerra`.
- Región: `us-east-1`.
- Block Public Access activado.
- Prefijos: `originals/` y `resized/`.

AWS Lambda

Propiedad	Valor propuesto	Justificación
Nombre	umg-image-resizer	Identifica claramente su responsabilidad.
Runtime	Node.js 22.x	Compatible con TypeScript compilado y Sharp.
Arquitectura	x86_64	Coincide con los binarios del ZIP.
Memoria	1024 MB	Valor inicial razonable para imágenes moderadas.
Timeout	30 segundos	Margen para descarga, proceso y carga.
Handler	index.handler	Exportación compilada en la raíz del ZIP.

Trigger

Event type: All object create events. **Prefix:** `originals/`. **Suffix:** vacío para admitir JPEG y PNG.

Dimensiones y Sharp

```
const pipeline = sharp(input, { failOn: "error" })
  .rotate()
  .resize({
    width: dimensions.width,
    height: dimensions.height,
    fit: "inside",
    withoutEnlargement: true,
  });
```

`fit: "inside"` coloca la imagen dentro del rectángulo 800 × 600 sin recortar ni deformar. `withoutEnlargement` evita ampliar entradas pequeñas. `rotate()` aplica la orientación EXIF antes del `resize`.

Resultados medidos localmente

1600×900

JPEG original
82 831 bytes

800×450

JPEG procesado
21 041 bytes

1200×1600

PNG original
76 423 bytes

450×600

PNG procesado
31 214 bytes

Las cifras corresponden a una ejecución local real de la misma estrategia Sharp. Los tamaños pueden variar ligeramente si cambia la versión o configuración del codificador.

Empaquetado para Lambda

El comando `npm run package:lambda` instala dependencias de producción con `--os=linux --cpu=x64 --libc=glibc`. El ZIP verificado contiene `sharp-linux-x64`, `libvips` y `index.js` en la raíz.

IAM de mínimo privilegio

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/originals/*"
},
{
  "Effect": "Allow",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/resized/*"
}
```

Controles aplicados

- No se concede **s3:***, borrado, administración ni lectura de otros prefijos.
- El bucket debe permanecer privado con bloqueo de acceso público.
- Lambda obtiene credenciales temporales a través de su execution role.
- Las claves de acceso, contraseñas y tokens no aparecen en el repositorio.
- Los permisos de logs se limitan al grupo de la función, excepto la creación inicial del grupo.
- La validación binaria reduce el riesgo de procesar un archivo disfrazado.

Auditoría local: la política JSON fue validada y no contiene acciones administrativas con comodín.

Consideraciones de producción

Para una solución pública también serían necesarios límites de tamaño, alarmas, presupuesto, retención de objetos y, si se utiliza una clave KMS propia, permisos KMS restringidos a esa clave.

Registro de la ejecución

El código emite logs estructurados con contexto suficiente para seguir cada registro sin imprimir el contenido del archivo o credenciales.

Momento	Información registrada
Inicio	Número de registros recibidos.
Evento	Bucket, key decodificada y nombre del evento.
Configuración	Content-Type esperado, ancho, alto y key de salida.
Descarga	Confirmación, bytes y Content-Type declarado por S3.
Sharp	Formato detectado, dimensiones iniciales/finales y bytes.
Guardado	Bucket, key final y Content-Type almacenado.
Error	Nombre, mensaje y stack para diagnóstico.

Interpretación de la captura simulada

La captura 11 reproduce exactamente la secuencia y nombres de campos programados. Los valores 1600×900, 800×450, 82 831 y 21 041 bytes proceden de la prueba local real; timestamp, RequestId y presentación de CloudWatch son simulados.

No es un log real de CloudWatch. Sirve para explicar qué se espera observar al desplegar.

Pruebas ejecutadas

Prueba	Entrada	Resultado	Estado
Compilación	TypeScript estricto	Sin errores ni variables sin utilizar	Correcto
Dependencias	<code>npm audit --omit=dev</code>	0 vulnerabilidades reportadas	Correcto
JPEG	1600×900, image/jpeg	800×450, image/jpeg	Correcto
PNG	1200×1600, image/png	450×600, image/png	Correcto
Decodificación	<code>mi+foto.jpg</code>	<code>mi foto.jpg</code>	Correcto
Salida anidada	<code>originals/viaje/Foto.JPEG</code>	<code>resized/viaje/Foto-resized.jpeg</code>	Correcto
Paquete Lambda	ZIP Linux x64	Binario Sharp y libvips presentes	Correcto
Integración AWS	S3 → Lambda → S3	No ejecutada por falta de acceso	Simulada

Conclusión de las pruebas

El núcleo de procesamiento y el artefacto de despliegue quedaron validados localmente. La ejecución integral en servicios AWS sigue pendiente y debe realizarse antes de afirmar que el sistema fue desplegado en la nube.

Controles de consistencia

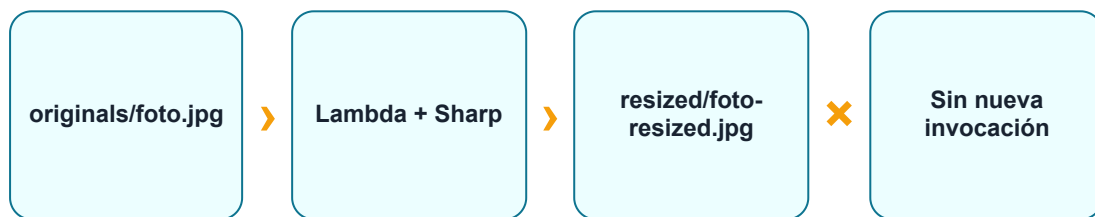
Prevención de ciclos

1. Filtro del trigger

S3 solo invoca Lambda por keys que comienzan con **originals/**. La salida se escribe en otro prefijo.

2. Defensa en código

El handler vuelve a comprobar el prefijo y termina el registro si recibe una key de **resized/** u otra ubicación.



Idempotencia

S3 puede entregar una notificación más de una vez. El mismo archivo siempre genera la misma key. Una repetición sobrescribe de forma segura el resultado anterior en vez de crear copias numeradas o nombres aleatorios.

```
originals/foto.jpg
↓
resized/foto-resized.jpg // siempre la misma salida
```

En una solución con control estricto del costo podrían registrarse key y eTag en DynamoDB con escritura condicional. Para el alcance académico, la sobrescritura determinista satisface la consistencia requerida.

Conclusiones

1. Una arquitectura basada en eventos desacopla la carga de la imagen de su transformación y elimina la necesidad de invocar manualmente el proceso.
2. Separar `originals/` y `resized/`, junto con una validación defensiva, evita ciclos de invocación.
3. Sharp conserva proporción y formato, pero sus binarios deben empaquetarse para el sistema operativo y arquitectura de Lambda.
4. Los nombres deterministas hacen que la entrega repetida de eventos produzca un estado consistente.
5. IAM y CloudWatch son parte funcional de la solución: reducen exposición y facilitan diagnóstico.

Repositorio

github.com/javier2002guerra-lab/Procesamiento-de-im-genes-con-arquitectura-serverless

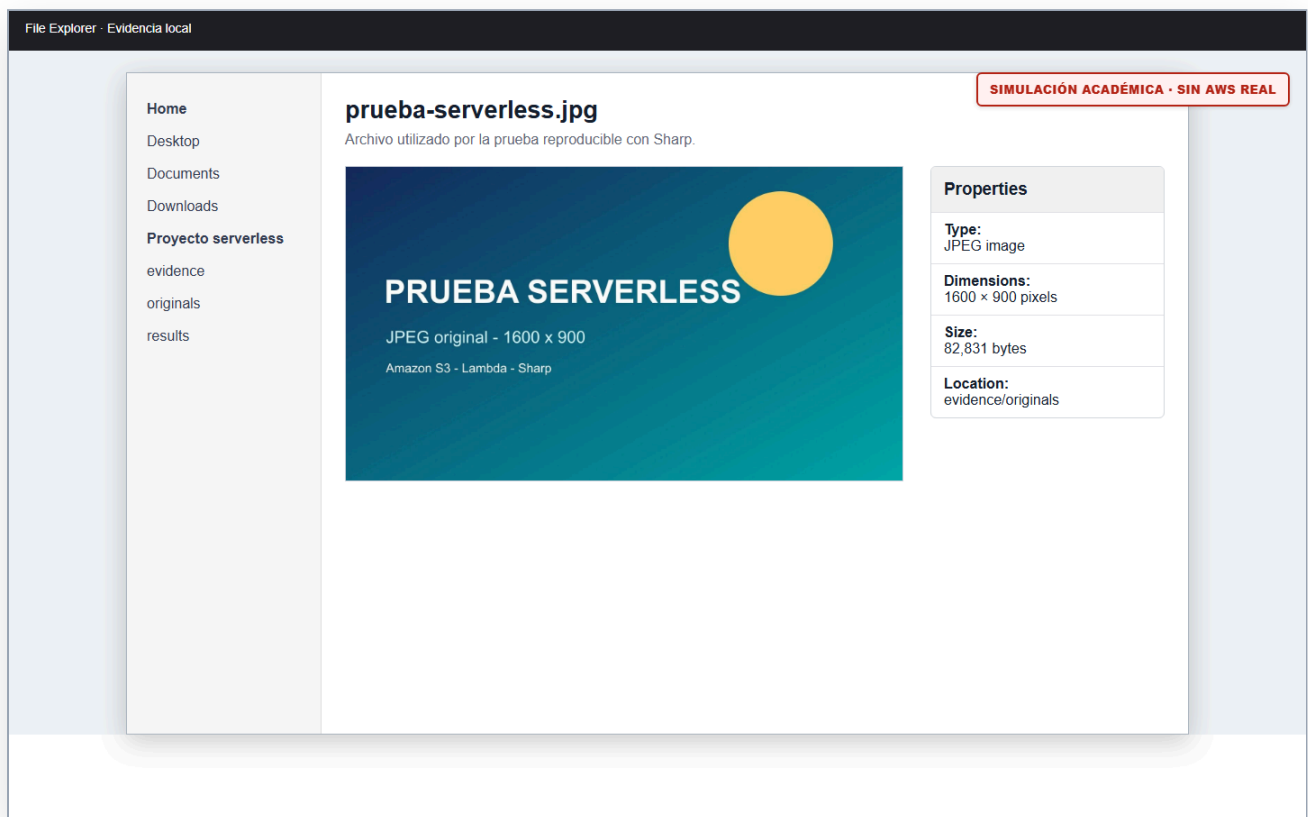
Referencias oficiales

- AWS. *Process Amazon S3 event notifications with Lambda*.
<https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html>
- AWS. *Building Lambda functions with Node.js*.
<https://docs.aws.amazon.com/lambda/latest/dg/lambda-nodejs.html>
- AWS. *Enabling and configuring event notifications using the Amazon S3 console*.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/enable-event-notifications.html>
- AWS. *Viewing CloudWatch logs for Lambda functions*.
<https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatchlogs-view.html>
- Sharp. *Installation — AWS Lambda*. <https://sharp.pixelplumbing.com/install/#aws-lambda>

Trabajo futuro: reemplazar las evidencias simuladas por capturas de una cuenta académica AWS y registrar métricas reales de duración y consumo.

SIMULACIÓN DECLARADA

Captura 1: Imagen original y dimensiones



Explicación: Archivo JPEG generado para una prueba reproducible. La imagen y sus dimensiones 1600x900 son reales y se verificaron localmente.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 2: Bucket S3

The screenshot displays the AWS S3 console interface. The top navigation bar includes the AWS logo, 'Services', a search bar, and the region 'N. Virginia' with a label 'Academic demo'. Below this, a dark blue header contains 'Amazon S3', 'Resource groups', 'Documentation', and 'Support'. A red-bordered box in the top right corner of the console area reads 'SIMULACIÓN ACADÉMICA • SIN AWS REAL'. The left sidebar shows the 'Amazon S3' menu with options: 'Dashboard', 'General purpose buckets' (highlighted), 'Access management', and 'Settings'. The main content area is titled 'Buckets' and shows 'Object storage resources in US East (N. Virginia)'. It includes a 'Refresh' button and a 'Create bucket' button. A table lists the buckets:

Bucket name	AWS Region	Access	Creation date
umg-serverless-images-demo-jguerra	US East (N. Virginia) us-east-1	Objects can be public: No	September 4, 2026

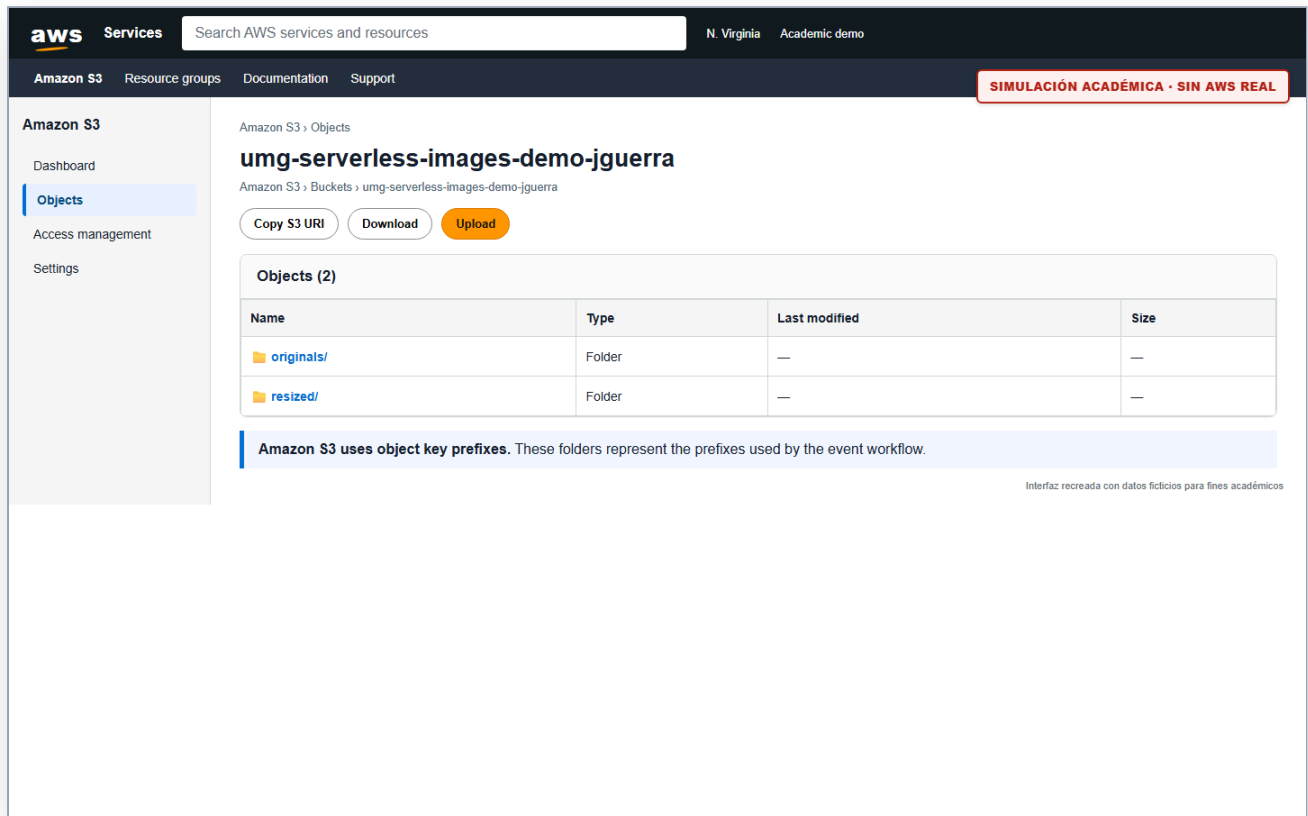
At the bottom right of the table, a small note states: 'Interfaz recreada con datos ficticios para fines académicos'.

Explicación: Recreación del bucket privado en us-east-1. Demuestra el nombre, región y bloqueo de acceso público esperados.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 3: Prefijos originals/ y resized/

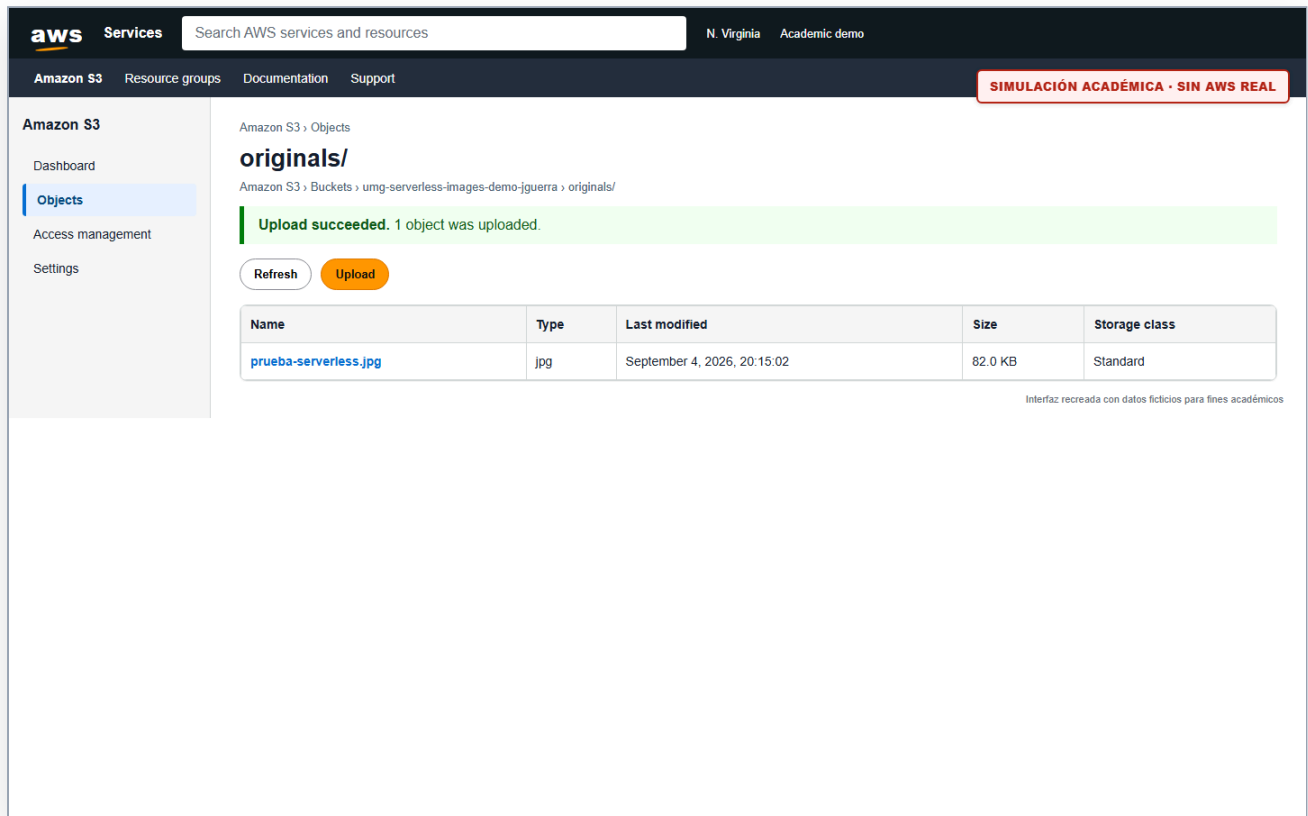


Explicación: Representación de los dos prefijos. originals/ recibe entradas y resized/ conserva resultados.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 4: Carga dentro de originals/



Explicación: Simulación del objeto creado en originals/. En AWS real esta operación generaría s3:ObjectCreated:.*.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 5: Función AWS Lambda

The screenshot displays the AWS Lambda console for the 'umg-image-resizer' function. The interface includes a navigation sidebar on the left with options like Dashboard, Functions, Access management, and Settings. The main content area shows the function's status as 'Active' and its description as a 'Serverless image processing function'. A diagram illustrates the workflow: Amazon S3 (umg-serverless-images-demo-jguerra) triggers the AWS Lambda function (umg-image-resizer), which then sends logs to Amazon CloudWatch. Below this, a 'Function overview' table provides technical details.

Function overview	
Runtime	Node.js 22.x
Architecture	x86_64
Handler	index.handler
Memory	1024 MB
Timeout	30 sec

Interfaz recreada con datos ficticios para fines académicos

Explicación: Recreación del resumen de umg-image-resizer con runtime, arquitectura, memoria, timeout y handler.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 6: Trigger de Amazon S3

The screenshot shows the AWS Lambda console interface. The top navigation bar includes the AWS logo, 'Services', a search bar, and the region 'N. Virginia' with a link to 'Academic demo'. Below this, a secondary navigation bar contains 'AWS Lambda', 'Resource groups', 'Documentation', and 'Support'. A red badge on the right of this bar reads 'SIMULACIÓN ACADÉMICA • SIN AWS REAL'. The left sidebar shows the 'AWS Lambda' menu with options: 'Dashboard', 'Triggers' (selected), 'Access management', and 'Settings'. The main content area is titled 'umg-image-resizer' and shows the 'Trigger configuration' for an Amazon S3 trigger. The configuration table is as follows:

Trigger	Amazon S3
Bucket	umg-serverless-images-demo-jguerra
Event types	All object create events (s3:ObjectCreated:*)
Prefix	originals/
Suffix	Not configured
Status	Enabled

Below the table, a green box states: 'Amazon S3 can invoke **umg-image-resizer** when an object is created under **originals/**.' At the bottom right, small text reads: 'Interfaz recreada con datos ficticios para fines académicos'.

Explicación: Configuración simulada de All object create events y destino Lambda.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 7: Filtro prefix originals/

The screenshot displays the AWS Management Console interface for configuring an Amazon S3 Event Notification. The left sidebar shows the navigation menu with 'Event notifications' selected. The main content area is titled 'image-upload-event' and shows the 'General configuration' section. The 'Event name' is 'image-upload-event', the 'Prefix' is 'originals/', and the 'Suffix' is 'No suffix configured'. A red box in the top right corner of the console indicates 'SIMULACIÓN ACADÉMICA • SIN AWS REAL'. A green message at the bottom states: 'Loop prevention: objects saved under resized/ do not match this prefix.'

Explicación: El prefijo con barra final constituye la barrera principal contra invocaciones por resized/.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 8: Variables de entorno

aws Services Search AWS services and resources N. Virginia Academic demo

AWS Lambda Resource groups Documentation Support

SIMULACIÓN ACADÉMICA - SIN AWS REAL

AWS Lambda

Dashboard

Environment variables

Access management

Settings

AWS Lambda › Environment variables

umg-image-resizer

Edit

Environment variables (2)

Key	Value
IMAGE_HEIGHT	600
IMAGE_WIDTH	800

The code reads these values through `process.env.IMAGE_WIDTH` and `process.env.IMAGE_HEIGHT`.

Interfaz recreada con datos ficticios para fines académicos

Explicación: IMAGE_WIDTH=800 e IMAGE_HEIGHT=600 mantienen las dimensiones fuera del código.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 9: Rol IAM

aws Services Search AWS services and resources N. Virginia Academic demo

IAM Resource groups Documentation Support

SIMULACIÓN ACADÉMICA - SIN AWS REAL

IAM

- Dashboard
- Roles**
- Access management
- Settings

IAM > Roles

umg-image-resizer-role

Execution role used by AWS Lambda.

Permissions policies

Policy name	Policy type	Used as
umg-image-resizer-policy	Customer inline	Permissions policy

Trust relationship

```
{
  "Effect": "Allow",
  "Principal": { "Service": "lambda.amazonaws.com" },
  "Action": "sts:AssumeRole"
}
```

Interfaz recreada con datos ficticios para fines académicos

Explicación: Recreación del execution role que Lambda asumiría mediante `lambda.amazonaws.com`.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 10: Política IAM

The screenshot displays the AWS IAM console interface. On the left, the 'IAM' menu is visible with options like 'Dashboard', 'Policies', 'Access management', and 'Settings'. The main content area shows the 'umg-image-resizer-policy' JSON policy document. The policy is a simulated one, as indicated by the red box in the top right corner of the screenshot that reads 'SIMULACIÓN ACADÉMICA - SIN AWS REAL'. The JSON document is as follows:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::umg-serverless-images-demo-jguerra/originals/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::umg-serverless-images-demo-jguerra/resized/*"
    },
    {
      "Effect": "Allow",
      "Action": "logs:CreateLogGroup",
      "Resource": "arn:aws:logs:*:*:*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:log-group:aws/lambda/umg-image-resizer:*"
    }
  ]
}
```

Interfaz recreada con datos ficticios para fines académicos

Explicación: Política simulada con GetObject en originals/*, PutObject en resized/* y permisos de logs.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 11: CloudWatch Logs

The screenshot displays the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, 'Services', a search bar, and the region 'N. Virginia' with the account name 'Academic demo'. The left sidebar shows the 'Amazon CloudWatch' menu with options for Dashboard, Log groups (selected), Access management, and Settings. The main content area shows the 'Log groups' page for the log group '/aws/lambda/umg-image-resizer'. The log stream is identified as 'Log stream 2026/09/04/[\$LATEST]academic-demo'. The log entries are simulated and show the execution of the 'umg-image-resizer' function, including details about S3 bucket operations, image processing with Sharp, and the final report. A red box in the top right corner of the console area reads 'SIMULACIÓN ACADÉMICA · SIN AWS REAL'. A small disclaimer at the bottom right of the log stream area states: 'Interfaz recreada con datos ficticios para fines académicos'.

```
2026-09-04T20:15:03.112Z START RequestId: demo-7b82 Version: $LATEST 2026-09-04T20:15:03.118Z Inicio de ejecución de umg-image-resizer. { "recordCount": 1 } 2026-09-04T20:15:03.120Z Registro S3 recibido. { "bucket": "umg-serverless-images-demo-jguerra", "originalKey": "originals/prueba-serverless.jpg", "eventName": "ObjectCreated:Put" } 2026-09-04T20:15:03.121Z Configuración de procesamiento. { "expectedContentType": "image/jpeg", "width": 800, "height": 600, "outputKey": "resized/prueba-serverless-resized.jpg" } 2026-09-04T20:15:03.282Z Descarga desde S3 confirmada. { "bytes": 82831, "declaredContentType": "image/jpeg" } 2026-09-04T20:15:03.491Z Procesamiento con Sharp confirmado. { "detectedContentType": "image/jpeg", "sourceDimensions": "1600x900", "outputDimensions": "800x450", "outputBytes": 21041 } 2026-09-04T20:15:03.598Z Guardado en S3 confirmado. { "outputKey": "resized/prueba-serverless-resized.jpg", "contentType": "image/jpeg" } 2026-09-04T20:15:03.599Z Fin de ejecución. { "processed": 1, "ignored": 0, "failed": 0 } 2026-09-04T20:15:03.601Z END RequestId: demo-7b82 2026-09-04T20:15:03.601Z REPORT Duration: 489.00 ms Memory Size: 1024 MB
```

Explicación: Log completamente simulado en apariencia y tiempo. Los nombres de mensajes coinciden con el código; dimensiones y bytes proceden de la ejecución local real.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 12: Resultado en resized/

aws Services Search AWS services and resources N. Virginia Academic demo

Amazon S3 Resource groups Documentation Support

Amazon S3

- Dashboard
- Objects
- Access management
- Settings

Amazon S3 > Objects

resized/

Amazon S3 > Buckets > umg-serverless-images-demo-jguerra > resized/

Refresh Download

Name	Type	Last modified	Size	Storage class
prueba-serverless-resized.jpg	jpg	September 4, 2026, 20:15:03	20.8 KB	Standard

The deterministic output key confirms the idempotent naming strategy.

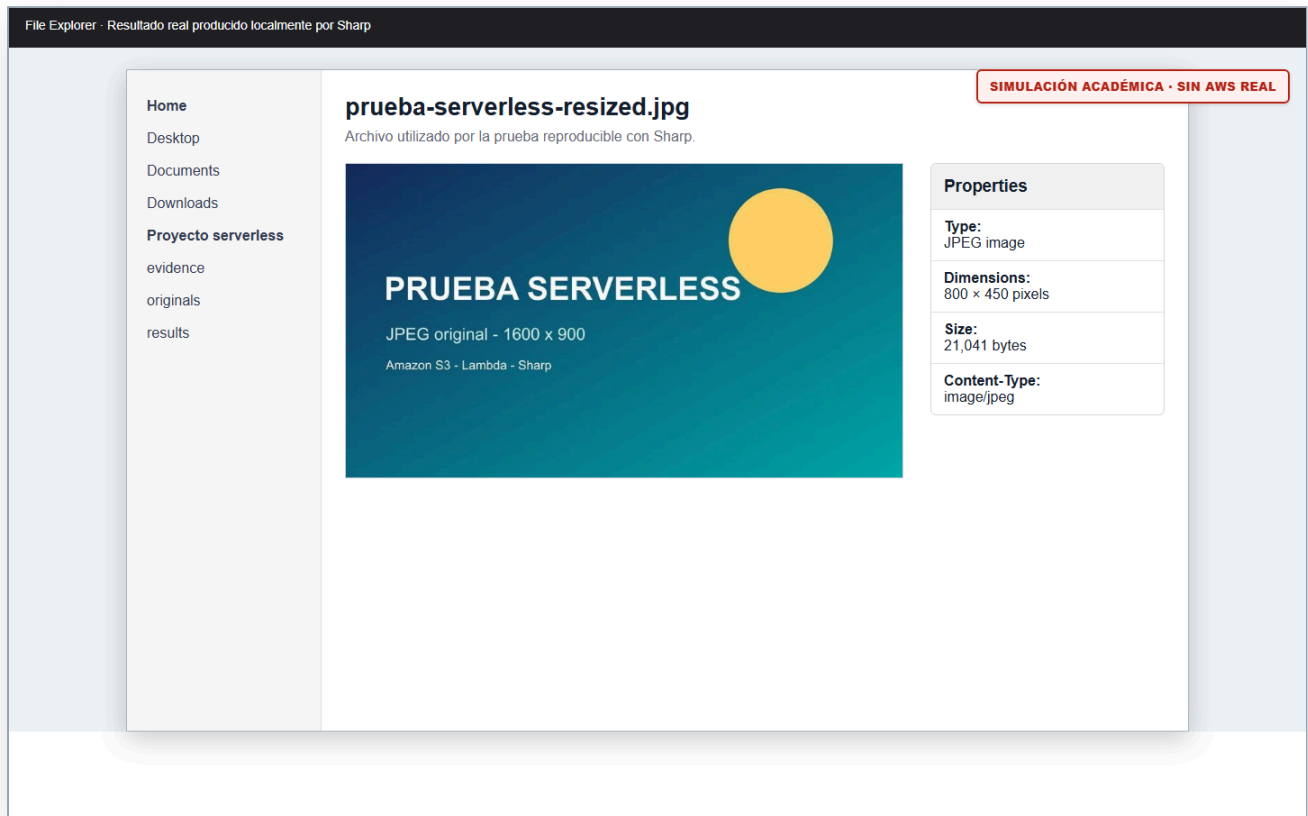
Interfaz recreada con datos ficticios para fines académicos

Explicación: Representación de la key determinista prueba-serverless-resized.jpg.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 13: Imagen procesada

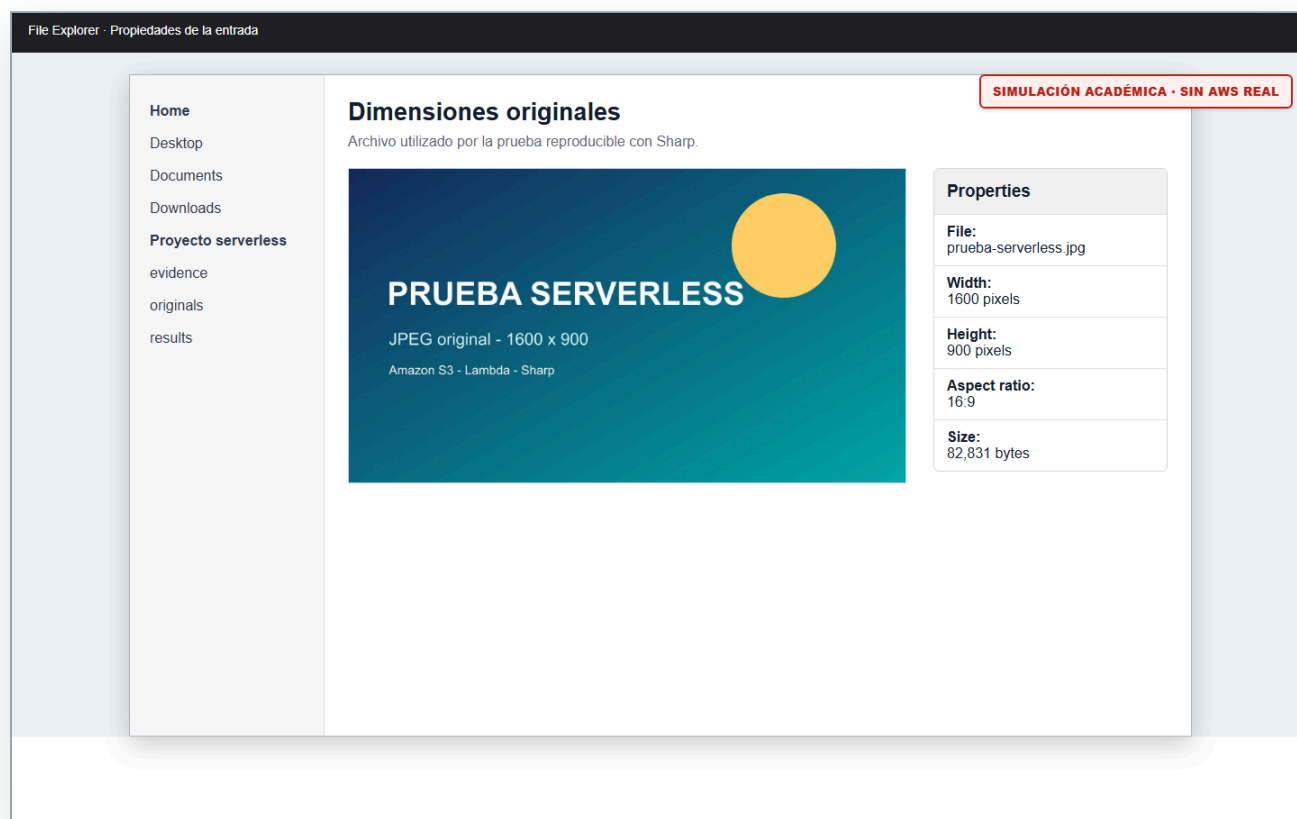


Explicación: Salida JPEG producida realmente por Sharp en la computadora, con 800×450 y Content-Type esperado image/jpeg.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 14: Dimensiones originales

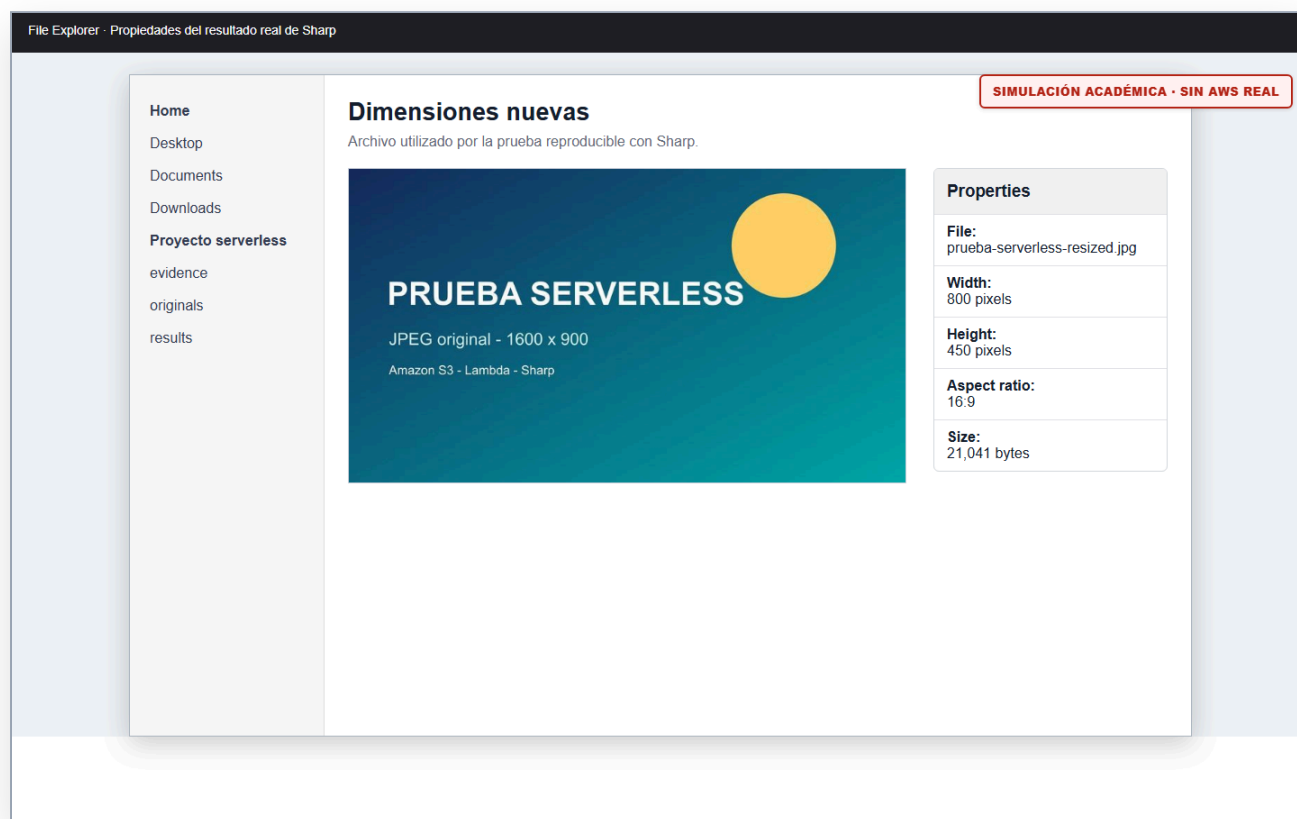


Explicación: Propiedades verificadas de la entrada real: 1600×900, relación 16:9.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 15: Dimensiones nuevas



Explicación: Propiedades verificadas del resultado real: 800×450, relación 16:9.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.

SIMULACIÓN DECLARADA

Captura 16: Comparación final

Comparación verificable · Procesamiento local real con Sharp

Original y procesada
La salida conserva formato JPEG y relación de aspecto 16:9.

SIMULACIÓN ACADÉMICA · SIN AWS REAL



Original · 1600 × 900 · 82,831 bytes



Procesada · 800 × 450 · 21,041 bytes

Verificación real de Sharp: el ancho se redujo 50 %, la altura se ajustó proporcionalmente y el original permanece separado.

Explicación: Comparación real local. Sharp redujo ambas dimensiones 50 %, conservó la proporción y mantuvo el original separado.

La marca roja dentro de la imagen identifica las pantallas simuladas y evita confundirlas con evidencia de una cuenta AWS activa.